

Programação para internet 1

Javascript
(Backend)

Prof. Luís Eduardo Tenório Silva
luís.silva@garanhuns.ifpe.edu.br

Elementos básicos de uma página Web

2

- **HTML**

- » Linguagem de marcação;
- » Define a estrutura de uma página.



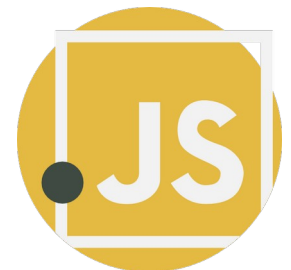
- **CSS**

- » Linguagem de estilização;
- » Define a aparência dos elementos de uma página.



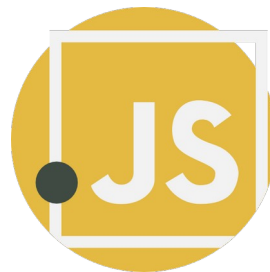
- **Javascript**

- » Linguagem de programação;
- » Define a interatividade de uma página;
- » Emite e trata eventos disparados pelo usuário.



O que é Back-end?

- Qualquer elemento no front-end pode ser manipulado pelo usuário (parte visual):
 - » Back-end abstrai partes que podem ser acessados pelo cliente de partes que somente a aplicação conhece.
- Processa solicitações vindas do cliente (um serviço, front-end, etc)
- Funções do back-end:
 - » Segurança
 - » Banco de dados
 - » Serviços externos
 - » Regras de negócio
 - » Validações de dados
 - » Logs e monitoramento



- API (Application Programming Interface)
 - » Comunicação entre sistemas diferente;
 - Sem necessidade de conhecimento de detalhes internos de implementação;
 - » Pode seguir um ou mais estilo arquitetural (modo como expõe os serviços para serem consumidos pelos sistemas):
 - REST (*Representational State Transfer*)
 - SOAP
 - RPC (*Remote Procedure Call*)
 - » Expõe pontos de acesso (rotas) que são acessados através de protocolos de comunicação (HTTP);

- Cliente-Servidor;
- Stateless;
- Utiliza o protocolo HTTP para comunicação entre cliente e servidor;
- Expõe recursos através de URI;
- Formatos comuns de mensagens:
 - » **JSON (JavaScript Object Notation)**
 - » XML (Extensible Markup Language)

Formatos de mensagem (XML)



```
1  <book>
2    <title>Harry Potter and the Sorcerer's Stone</title>
3    <author>J.K. Rowling</author>
4    <genre>Fantasy</genre>
5    <price>19.99</price>
6  </book>
```

Formatos de mensagem (JSON)



```
1  {  
2      "title": "Harry Potter and the Sorcerer's Stone",  
3      "author": "J.K. Rowling",  
4      "genre": "Fantasy",  
5      "price": 19.99  
6  }
```

• Requisição HTTP

```
GET /somedir/page.html HTTP/1.1 } Linha de requisição  
Host: www.someschool.edu  
Connection: close  
User-agent: Mozilla/5.0  
Accept-language: fr
```

• Linha de requisição

- » Método: Ação a realizar (GET, POST, HEAD, PUT, DELETE)
- » Objeto requisitado: /somedir/page.html
- » Versão do HTTP: http/1.1

• Requisição HTTP

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

} Linhas de cabeçalho

• Linhas de cabeçalho

- » Host: endereço do servidor
- » Connection: persistente(*keep-alive*), não persistente (*closed*)
- » User-agent: browser/cliente
- » Accept-language: Linguagem preferencial (se houver) do conteúdo

- **Resposta HTTP**

```
HTTP/1.1 200 OK } Linha de estado
Connection: close
Date: Tue, 09 Aug 2011 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 09 Aug 2011 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
```

(dados dados dados dados dados ...)

- **Linha de estado**

- » Versão: Versão do protocolo HTTP
- » Código de estado: Código de resposta da requisição HTTP
- » Frase: mensagem correspondente ao código de estado

- Agrupados em 5 classes (RFC 2616):
 - » **100-199**: Respostas de informação
 - » **200-299**: Respostas de sucesso
 - » **300-399**: Redirecionamentos
 - » **400-499**: Erros do cliente
 - » **500-599**: Erros do servidor
- Acessíveis através deste [link](#).

Respostas de sucesso (200-299)

12

- **200 OK:** Requisição bem sucedida
- **201 Created:** Requisição bem sucedida e novo recurso criado.
- **202 Accepted:** Nenhuma ação foi tomada sobre a requisição
- **204 No Content:** Não existe um conteúdo para ser encaminhado ao cliente.
- **206 Partial Content:** Separa o conteúdo em vários fluxos.

- **301 Moved Permanently**: Objeto foi movido para outra URL (Definido no cabeçalho *Location* da mensagem de resposta)

Resposta de erro do cliente (400-499)

14

- **400 Bad Request**: Erro genérico. Requisição do cliente não está como esperado pelo servidor.
- **401 Unauthorized**: O cliente não está autenticado.
- **403 Forbidden**: O cliente não tem direitos de acesso ao conteúdo (autenticado).
- **404 Not Found**: Recurso requisitado não existe no servidor.
- **405 Method Not Allowed**: Método utilizado pelo cliente não é permitido (PUT, POST, DELETE...)

Resposta de erro do servidor (500-599)

15

- **500 Internal Server Error**: Erro genérico.
- **502 Bad Gateway**: Erro de comunicação com *gateway* do servidor.
- **503 Service Unavailable**: O servidor está sobrecarregado e não pode atender a requisição. Cabeçalho *Retry-After* deverá dizer o tempo para usuário retentar.
- **505 HTTP Version Not Supported**: Versão do protocolo HTTP não suportada pelo servidor.

- Característica atribuída a serviços web que seguem os princípios e regras de uma arquitetura REST
- Princípios de uma arquitetura REST
 - » Separação do cliente e servidor;
 - » Sem armazenamento de estado do cliente;
 - » Todo recurso deve ser representado por um ID;
 - » Operações padronizadas;
 - » Definição da representação dos recursos;
 - » Utilização de hipermídia para vinculação de recursos (HATEOAS);

- Obtendo todos os recursos

Verbo HTTP: GET, POST

URL: `http://localhost:3000/filmes`

» Resposta:

· `[{id: 1, nome:'filme 1', ano: 1999}, {id: 2, nome:'filme 2', ano: 2000},
{id: 3, nome:'filme 3', ano: 1975}]`

- Cadastrar um novo recurso

Verbo HTTP: POST

URL: `http://localhost:3000/filmes`

» Body da requisição

```
{ nome: "Filme Novo", ano: 2022 }
```

- Atualizar um recurso

Verbo HTTP: PUT, PATCH

URL: `http://localhost:3000/filmes/1`

» Body da requisição

```
{ ano: 2015 }
```

- Remover um recurso

Verbo HTTP: DELETE

URL: <http://localhost:3000/filmes/1>

- Ambiente de execução JS no lado servidor;
- Construído sobre o motor v8:
 - » Interpretador Javascript;
 - » Utilizado no Google Chrome.
- Características:
 - » Assíncrono;
 - » Utilização de módulos e pacotes;
 - » Escalável;



Implementando uma API REST

- Criar um novo diretório
 - » *mkdir meuingresso-backend*
 - » *cd meuingresso-backend*
- Criar um projeto node
 - » *npm init -y*
- Instalar dependências de desenvolvimento
 - » *npm i -D nodemon*
- Instalar dependências da aplicação
 - » *npm i express*

package.json

23



```
1  {
2    "name": "meuingresso-backend",
3    "version": "1.0.0",
4    "description": "Backend do meuingresso",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": ["filmes", "promocoes", "brasil"],
10   "author": "Luís Eduardo",
11   "license": "ISC",
12   "devDependencies": {
13     "nodemon": "^3.1.4"
14   },
15   "dependencies": {
16     "express": "^4.19.2"
17   }
18 }
19
```

- Criar uma pasta na raiz denominada public
 - » *mkdir public*
- Copiar todos os arquivos da página web para dentro da pasta public

meuingresso-backend/

```
├─ public/
│   ├─ css/
│   │   └─ style.css
│   ├─ js/
│   │   └─ script.js
│   └─ images/
│       └─ logo.png
└─ index.html
└─ app.js
```

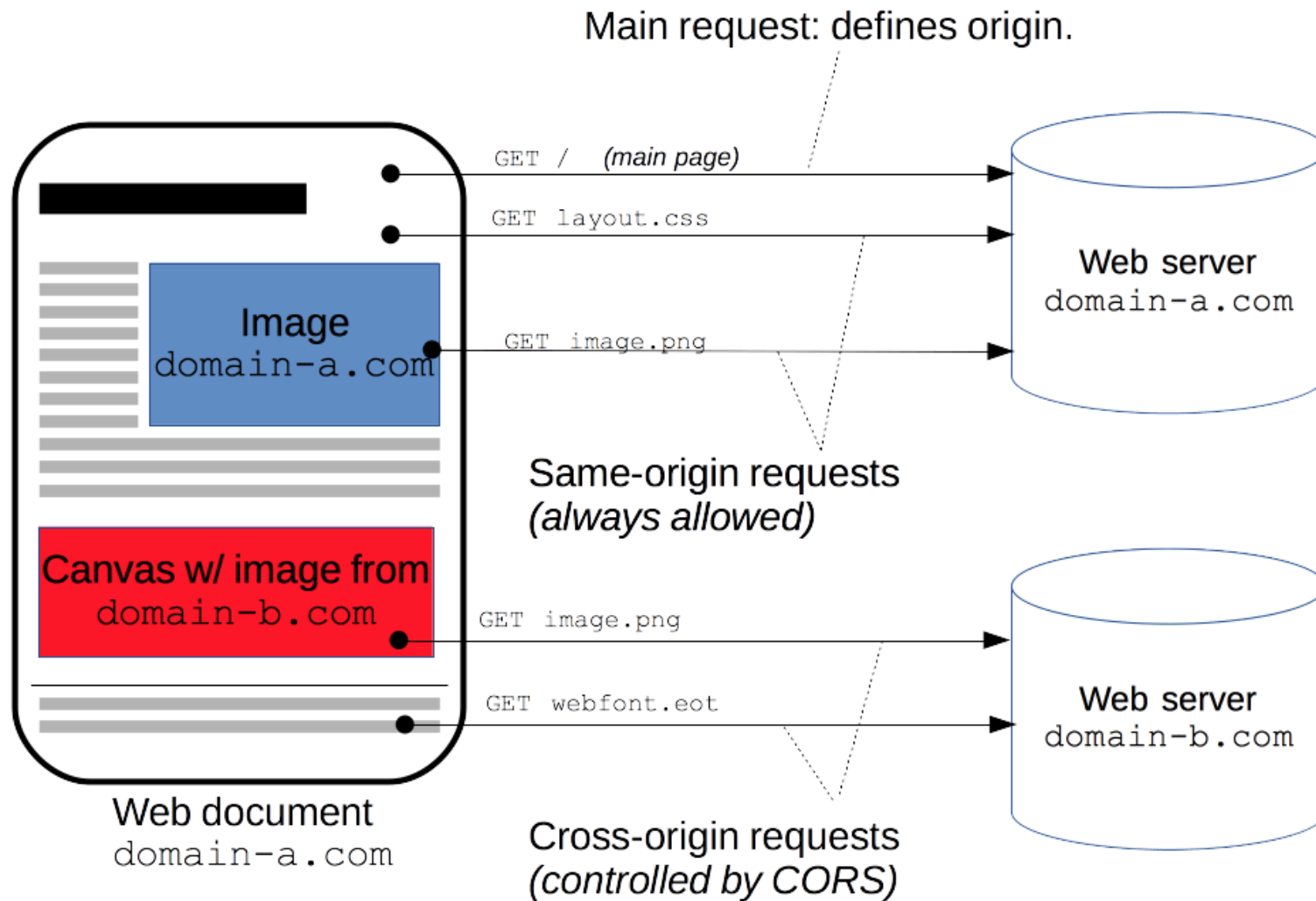

Prover arquivos estáticos

25



```
1  import express from "express";
2
3  const app = express()
4
5  app.use(express.static('public'))
6
7  const porta = 3000;
8
9  app.listen(porta, () => {
10     console.log(`Servidor rodando na porta ${porta}`);
11 });
```

- *Cross Origin Resource Share*
- Recurso de segurança aplicado;
 - » Utiliza o cabeçalho HTTP
 - » Informa os navegadores quais recursos podem ser acessados em um determinado domínio;
- Ex:
 - » Domínio: `http://dominio-a.com` hospeda aplicação (HTML, CSS, JS)
 - » Se JS (frontend) tenta buscar dados em `http://dominio-b.com/dados.json`
 - Navegador restringem requisição HTTP para domínios diferentes (CORS);
 - Backend informa ao frontend (no momento da obtenção da aplicação, via HTTP Header response) que pode buscar dados em outro domínio (políticas de acesso).
- Controla no navegador as solicitações e respostas de diferentes origens;
- O backend precisa apontar quais os domínios frontends que ele responderá;
- No backend:
 - » `npm i cors`



Cross Origin Resource Share

Fonte: MDN



```
1  import cors from "cors";  
2  
3  const app = express()  
4  
5  // Configuração da API  
6  app.use(cors());
```



```
1 import express from "express";
2
3 const app = express()
4
5 // Configuração da API
6 app.use(express.json());
```



```
1 export const filmes = [  
2   { id: 1, nome: 'Filme 1' },  
3   { id: 2, nome: 'Filme 2' },  
4   { id: 3, nome: 'Filme 3' },  
5 ];  
6
```



```
1 app.get('/api/filmes', (req, res) => {  
2   return res.status(200).json(filmes)  
3 })
```

- Sistema de gerenciamento de banco de dados (SGBD);
- Relacional;
- SQL (Structure Query Language);
- Leve e portátil;
 - » Dados salvos em um único arquivo.
- Zero configuração:
 - » Dispensa configurações de administração
 - Instalação, configuração de usuários e permissões, etc.
- Multiplataforma.



- Criar um diretório na raiz do projeto chamado *database*;
 - » *mkdir database*
- Criar um arquivo *db.js* e um arquivo *seed.js* dentro da pasta *database*;
 - » *touch database/db.js* → Descrição das tabelas do BD;
 - » *touch database/seed.js* → Entradas iniciais do BD.
- Instalar os pacotes *sqlite3*
 - » *npm i sqlite3 sqlite*
- Instalar plugin VSCode
 - » SQLite

- Definindo conexão



```
1 import { open } from "sqlite";
2 import sqlite3 from "sqlite3";
3
4 const dbPromise = open({
5   filename: "./database/filmes.db",
6   driver: sqlite3.Database,
7 });
```

- Realizando conexão



```
1 const db = await dbPromise;
```

- Executando uma query:

```
1 const filmes = await db.all('SELECT * FROM filmes');
```

- Inserindo elemento no BD:

```
1 await db.run(  
2     "INSERT INTO filmes (id, titulo, img, descricao, genero, diretor, valor, ano) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",  
3     [  
4         id,  
5         filme.titulo,  
6         filme.img,  
7         filme.descricao,  
8         filme.genero,  
9         filme.direcao,  
10        filme.valor,  
11        filme.ano,  
12    ]  
13 );
```

- **db.run(sql, [params], [callback]):** Executa uma instrução SQL (INSERT, UPDATE, DELETE)
 - » params: parâmetros utilizados na inserção (opcional). Array;
 - » Callback: função executada após a execução do comando (opcional).

```
1  await db.run(  
2    "INSERT INTO filmes (id, titulo, img, descricao, genero, diretor, valor, ano) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",  
3    [  
4      id,  
5      filme.titulo,  
6      filme.img,  
7      filme.descricao,  
8      filme.genero,  
9      filme.direcao,  
10     filme.valor,  
11     filme.ano,  
12   ]  
13  );
```

- **db.get(sql, [params], [callback]):** Obtém apenas a primeira linha do resultado de uma consulta;
 - » params: parâmetros utilizados na inserção (opcional). Array;
 - » Callback: função executada após a execução do comando (opcional).



```
1 const filmes = await db.all('SELECT * FROM filmes');
```

- **db.all(sql, [params], [callback]):** Obtém todas as linhas da execução de uma consulta.
 - » params: parâmetros utilizados na inserção (opcional). Array;
 - » Callback: função executada após a execução do comando (opcional).



```
1 const filmes = await db.all('SELECT * FROM filmes');
```



• Tabela Filmes

- » id:
 - Tipo: string;
 - Chave primária;
 - Ex: "d3136953-7ba1-4929-a08f-2727d20e5238"
- » img:
 - Tipo: string
 - Ex:
"https://4.bp.blogspot.com/-GW1Ex5rl1o8/VMG9B8zuM2I/AAAAAAAAAQ-A/PdXFAKUIN2c/s1600/somaoredor1.jpeg"
- » titulo:
 - Tipo: string;
 - Ex: "O Som ao redor".
- » descricao:
 - Tipo: string;
 - Ex: "O Som ao redor ...".
- » ano:
 - Tipo: inteiro;
 - Ex: 2013.
- » direcao:
 - Tipo: string;
 - Ex: "Kleber Mendonça".
- » genero:
 - Tipo: string;
 - Ex: "Suspense, Drama".
- » valor:
 - Tipo: float;
 - Ex: 3.50.

```
CREATE TABLE IF NOT EXISTS  
filmes (
```

```
    id TEXT PRIMARY KEY,
```

```
    titulo TEXT,
```

```
    img TEXT,
```

```
    descricao TEXT,
```

```
    genero TEXT,
```

```
    direcao TEXT,
```

```
    ano INTEGER,
```

```
    valor REAL
```

```
)
```

• Tabela Usuários

» id:

- Tipo: inteiro;
- Chave primária;
- Pode ser gerado pelo BD;
- Ex: 15.

» nome:

- Tipo: string;
- Único;
- Não nulo;
- Ex:
"luis.silva@garanhuns.ifpe.edu.br,".

» senha

- Tipo: string;
- Não nulo;
- Ex: "12345".

```
CREATE TABLE IF NOT EXISTS  
users (
```

```
    id INTEGER PRIMARY KEY  
    AUTOINCREMENT,
```

```
    nome TEXT UNIQUE NOT NULL,
```

```
    senha TEXT NOT NULL
```

```
)
```


• Tabela vendas

» Id

- Tipo: inteiro;
- Chave primária;
- Pode ser gerado pelo BD;
- Ex: 2.

» id_usuario:

- Tipo: inteiro;
- Chave estrangeira tabela usuarios;
- Ex: 15.

» id_filme:

- Tipo: string;
- Chave estrangeira tabela filmes;
- Ex: "d3136953-7ba1-4929-a08f-2727d20e5238".

```
CREATE TABLE IF NOT EXISTS  
vendas (
```

```
    id INTEGER PRIMARY KEY  
    AUTOINCREMENT,
```

```
    id_usuario INTEGER,
```

```
    id_filme TEXT,
```

```
    FOREIGN KEY(id_usuario)  
    REFERENCES usuarios(id),
```

```
    FOREIGN KEY(id_filme)  
    REFERENCES filmes(id)
```

```
)
```

- Arquivo *database/db.js*

```
1 import sqlite3 from "sqlite3";
2 import { open } from "sqlite";
3
4 const dbPromise = open({
5   filename: "./database/filmes.db",
6   driver: sqlite3.Database,
7 });
8
9 async function createTableFilmes() {
10   const db = await dbPromise;
11   await db.exec(`
12     CREATE TABLE IF NOT EXISTS filmes (
13       id TEXT PRIMARY KEY,
14       titulo TEXT,
15       img TEXT,
16       descricao TEXT,
17       genero TEXT,
18       diretor TEXT,
19       ano INTEGER,
20       valor REAL
21     )
22   `);
23 }
24
25 createTableFilmes().catch((err) => console.error(err));
26
27 export { dbPromise };
```

- **Driver nativo**
- **Query Builder**
- **Object-Relational Mapper (ORM)**

- **Driver nativo**

- » Mais próximo ao BD;
- » Exige escritas das Queries
 - `SELECT * FROM exemplo WHERE campo="valor";`
- » Vantagens:
 - Maior desempenho.
 - Programador pode otimizar as escritas das queries;
- » Desvantagens:
 - Complexidade;
 - Exige conhecimento de SQL
 - Controle de conexão;
 - Problemas de segurança quando não manipulado corretamente;
 - SQL Injection

- **Driver nativo**

- » Ex (bibliotecas):

- MySQL: mysql, mysql2;
 - PostgreSQL: pg;
 - SQLite: sqlite, sqlite3.

- Driver nativo



```
1  import sqlite3 from 'sqlite3';
2  sqlite3.verbose();
3
4  const db = new sqlite3.Database('database_name.db');
5
6  db.serialize(() => {
7    db.each('SELECT * FROM table_name', (err, row) => {
8      if (err) throw err;
9      console.log(row);
10    });
11  });
12
13  db.close();
```

• Query Builder

- » Utiliza recursos de programação (funções) para construir uma query que será executada em um BD;
 - Um grupo de funções executadas sequencialmente para representar um comando SQL.
- » Não exige a escrita de queries;
 - Mas exige o conhecimento de como elas são construídas.

• Query Builder

» Vantagens:

- Simplicidade no uso do BD;
- Trata nativamente problemas de segurança como SQL Injection;

» Desvantagens

- Menos possibilidade de otimização das queries.
 - Menos performance;

» Ex (biblioteca):

- knex.

- Query Builder



```
1 import knex from 'knex';
2
3 const db = knex({
4   client: 'mysql',
5   connection: {
6     host: '127.0.0.1',
7     user: 'your_database_user',
8     password: 'your_database_password',
9     database: 'myapp_test'
10  }
11 });
12
13 db.select('*').from('users').then(rows => {
14   console.log(rows);
15 }).catch(err => {
16   console.log(err);
17 }).finally(() => {
18   db.destroy();
19 });
```

- **Object-Relational Mapper (ORM)**

- » Mapeamento de um elemento relacional de um BD em um objeto que pode ser manipulado pela linguagem de programação;
- » Não exige escrita de queries;
 - Pouco conhecimento de como são construídas.
- » Maior nível de abstração;
 - Abstrai do usuário consultas, armazenamento, edições e exclusões de elementos do BD.

• Object-Relational Mapper (ORM)

» Vantagens:

- Facilidade para o desenvolvedor;
- Atividades complexas realizadas de maneira simples;
 - Migrações (versionamento de BD)
- Linguagem agnóstica ao banco.

» Desvantagens:

- Overhead em consultas;
- Baixo desempenho;
- Curva de aprendizado complexa.

» Exemplos (bibliotecas):

- TypeORM, sequelize, Hibernate.

- **Object-Relational Mapper (ORM)**

```
1 import { Sequelize, DataTypes } from "sequelize";
2
3 const sequelize = new Sequelize("database", "username", "password", {
4   host: "localhost",
5   dialect: "mysql", // ou 'postgres', 'sqlite', 'mssql'
6 });
7
8 const User = sequelize.define("User", {
9   username: {
10     type: DataTypes.STRING,
11     allowNull: false,
12   },
13   birthday: {
14     type: DataTypes.DATE,
15     allowNull: false,
16   },
17 });
18
19 (async () => {
20   await sequelize.sync();
21   const jane = await User.create({
22     username: "janedoe",
23     birthday: new Date(1980, 6, 20),
24   });
25   console.log(jane.toJSON());
26 })();
```

- UUID: Universally Unique Identifier
- Identificação de qualquer elemento em computação;
- Único em um sistema distribuído;
- 128 bits
 - » Representado por 32 dígitos hexadecimais:
 - 8-4-4-12
 - » Ex: c5b9b859-caa4-4fc7-932e-40cb1168e41b
- Instalar os pacotes uuid
 - » `npm i uuid`



```
1 import { v4 as uuidv4 } from "uuid";  
2 const id = uuidv4();  
3  
4 console.log(id);
```



```
1 routerLogin.post("/api/login", (req, res) => {
2   const { usuario, senha } = req.body;
3   if (usuario && senha) {
4     return res.status(200).json({ message: "OK!" });
5   } else {
6     return res.status(400).json({ message: "Usuário ou senha inválidas" });
7   }
8 });
```

- **bcryptjs** (segurança)

- » Descrição: Biblioteca js para criptografar senhas;
- » Propósito: Armazenamento seguro de senhas;
- » Gera um hash de uma senha
 - O hash é armazenado no banco ao invés da senha.
- » Instalação:
 - *npm i bcryptjs*



```
1 import bcrypt from 'bcryptjs';
2
3 const password = 'myPassword';
4 const salt = await bcrypt.genSalt(10);
5 const hashedPassword = await bcrypt.hash(password, salt);
6 console.log(hashedPassword);
```


- Geração de hash:



```
1 import bcrypt from 'bcryptjs';
2
3 const password = 'myPassword';
4 const salt = await bcrypt.genSalt(10);
5 const hashedPassword = await bcrypt.hash(password, salt);
6 console.log(hashedPassword);
```

- Validação de hash:



```
1 const isMatch = await bcrypt.compare(password, hashedPassword);
2 if (isMatch) {
3   console.log('Senha válida');
4 } else {
5   console.log('Senha inválida');
6 }
```

- **jsonwebtoken**

- » Descrição: Biblioteca js para criar e verificar tokens JWT;
- » Propósito: Autenticar usuários e autorizar acesso a recursos;
- » Instalação:
 - *npm i jsonwebtoken*

- Criação de tokens

- » Gera um token e o assina com uma chave privada.
 - Caso qualquer caractere seja mudado, na verificação.
- » Inclui informações no payload como identificação do usuário e permissões.

- **Estrutura do JWT (separado por .)**

- » **Header**: Tipo do token e algoritmo de assinatura utilizado
- » **Payload**: Informações (claims) que o backend deseja enviar:
 - Ex: userID, role, exp (tempo de expiração)
- » **Signature**: Assinatura do header e do payload para garantir integridade.

- Header e payload são encodados em base64 para minimizar o tamanho do token;

- Ex:

- » eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTYyMzkwMjAwLCJleHBpcmVhdGlzbiI6MTU2MjM5MDMwMH0.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJxMjM0NSIsIm5vbWUiOiJMdc0tcyBFZHVhcmRvIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzIyNDUxMTczLCJleHAiOjE3MjI0NTExNzR9.mwajQaeEEUqla6Fg4bMX7kqU5QSo-2vcCa3Vse4paac
```

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "userId": "12345",
  "nome": "Luis Eduardo",
  "role": "admin",
  "iat": 1722451173,
  "exp": 1722451174
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  3cret@!n@o-c0mp@rt!h@r
) ☐ secret base64 encoded
```

✓ Signature Verified

SHARE JWT

- Criação do token



```
1 import jwt from 'jsonwebtoken';  
2  
3 const payload = { userId: '12345', nome: "Luís Eduardo", role: 'admin' };  
4 const secret = 'chave-sup3r-s3cret@!n@o-c0mp@rt!lh@r';  
5 const token = jwt.sign(payload, secret, { expiresIn: '10m' });  
6 console.log('Token JWT:', token);
```

- Validação do token



```
1 try {  
2   const decoded = jwt.verify(token, secret);  
3   console.log('Token válido:', decoded);  
4 } catch (error) {  
5   console.log('Token inválido:', error.message);  
6 }
```


Rotas DELETE

Dúvidas?